UNIVERSIDAD EAN

Bogotá, Colombia

Learning Activity 3

DATA INTELLIGENCE CHALLENGE

Cuatro desafíos de ETL, indicadores y decisiones con datos de NovaMarket, VidaPlus y RutaExpress

Estudiante: Jorge Piñeros

Sistemas de información

Tercer periodo — 25 de octubre de 2026

Tabla de contenido

Introducción y objetivo 3
Desarrollo del laboratorio 3
Ejercicio 1 — NovaMarket: ingresos y calidad de ventas (Power BI) 4
Ejercicio 2 — VidaPlus: oportunidad del servicio (Power BI) 7
Ejercicio 3 — RutaExpress: retrasos y costos (Python) 9
Ejercicio 4 — NovaMarket: clientes y fidelización (Python) 13
Tabla comparativa de hallazgos y calidad de datos 18
Tres respuestas sobre el proceso ETL 19
Conclusiones y recomendaciones 21
Fuentes de información y bibliografía 22
Anexos 23
Anexo A. Código de Power Query (M) y medidas DAX 23
Anexo B. Código Python (notebook) 25
Anexo C. Bitácora de cambios de datos 30
Anexo D. Archivos depurados y entregados 31

Introducción y objetivo

Este informe presenta el desarrollo del Data Intelligence Challenge. Se trabajaron cuatro conjuntos de datos de tres organizaciones: **NovaMarket** (comercio minorista, dos archivos), **VidaPlus** (salud) y **RutaExpress** (logística). Cada archivo tiene 36 registros, para un total de 144, y todos traen anomalías puestas a propósito que hay que encontrar y documentar.

Objetivo: aplicar procesos ETL (extraer, transformar y cargar) para preparar, analizar y visualizar datos con Power BI y Python, calcular indicadores que se puedan verificar y justificar decisiones de gestión con cifras concretas.

La forma de trabajo fue la misma en los cuatro ejercicios: (1) importar el CSV y capturar el estado original, (2) estandarizar textos, validar rangos y faltantes y marcar las anomalías con un criterio explícito, sin inventar valores, (3) cargar los datos preparados y (4) calcular indicadores, graficar y tomar una decisión. Los CSV originales no se modificaron; todo lo transformado se guardó en archivos nuevos.

Los ejercicios 1 y 2 se hacen en Power BI Desktop y los ejercicios 3 y 4 en Python (pandas y matplotlib). Para los ejercicios 1 y 2 las cifras de este informe se verificaron además con pandas, de modo que las tarjetas del tablero deben dar los mismos valores.

Datos de partida

Archivo	Organización	Registros	Columnas	Herramienta
01_NovaMarket_Ventas.csv	NovaMarket (retail)	36	9	Power BI
02_VidaPlus_Atenciones.csv	VidaPlus (salud)	36	7	Power BI
03_RutaExpress_Envios.csv	RutaExpress (logística)	36	7	Python
04_NovaMarket_Clientes.csv	NovaMarket (retail)	36	6	Python

Nota: el zip traía además un archivo llamado «02_VidaPlus_Atenciones Copiar.csv». Lo comparé con el original (comando diff) y es idéntico, así que no se usó y no cambia ningún resultado.

Desarrollo del laboratorio

Cada ejercicio tiene su enunciado, el proceso ETL, los indicadores, las visualizaciones y la decisión gerencial. Las capturas (1 a 8) son la salida real del código de Python ejecutado en el notebook de los ejercicios 3 y 4.

Cómo leer este informe

- **Gráficos (Figuras):** generados con Python a partir de los datos ya depurados. En los ejercicios 1 y 2 sirven como referencia para comprobar que el tablero de Power BI muestra las mismas cifras.
- **Símbolo ␣:** en los textos con errores de formato representa un espacio en blanco (por ejemplo «␣norte␣» = « norte »).
- **Capturas:** salida real del código de Python de los ejercicios 3 y 4, numeradas con el código de la guía (E3.1 a E4.4).
- **Criterio general de limpieza:** nunca se inventan valores. Lo faltante o inválido queda vacío (null/NaN), se marca en una columna aparte y se excluye solo del cálculo que lo necesita; ninguna fila se elimina.

Ejercicio 1 — NovaMarket: ingresos y calidad de ventas (Power BI)

Enunciado. La gerencia de NovaMarket necesita identificar los productos, regiones y canales de mayores ingresos y verificar errores de facturación. Archivo: 01_NovaMarket_Ventas.csv (36 ventas del 1 al 28 de septiembre de 2026).

E — Extraer

En Power BI Desktop: Obtener datos → Texto/CSV → seleccionar el archivo → Transformar datos. Se importan **36 filas y 9 columnas**: ID_Venta, Fecha, Region, Canal, Producto, Unidades, Precio_Unitario, Descuento_Pct e Ingreso_Registrado. Hay 4 productos (Teclado, Monitor, Mouse, Audífonos), 3 regiones y 3 canales.

T — Transformar

#	Transformación en Power Query	Qué corrige
1	Region y Canal: Recortar + Poner en mayúscula cada palabra	V009 «␣norte␣» → Norte; V028 «web␣» → Web. Sin esto Power BI crea categorías separadas.
2	Tipos de dato: Fecha → fecha; Unidades, Precio_Unitario y Descuento_Pct → entero; Ingreso_Registrado → decimal	Garantiza que los cálculos sean numéricos.
3	Columna personalizada Ingreso_Esperado = [Unidades] * [Precio_Unitario] * (1 - [Descuento_Pct] / 100)	Ingreso que debería tener cada venta.
4	Columna personalizada Diferencia = [Ingreso_Registrado] - [Ingreso_Esperado]	Permite detectar errores de facturación.

Resultado de la validación: no hay faltantes ni valores fuera de rango en las columnas numéricas. Las anomalías de este archivo son **3**: dos de formato de texto (V009 y V028) y una de facturación (V020, ver indicadores).

L — Cargar

Con Cerrar y aplicar la tabla se carga al modelo (renombrada como «Ventas»). Sobre ella se crean las medidas DAX del Anexo A y los visuales.

Indicadores

Indicador	Resultado
Ingreso esperado total	$12.723.750
Ingreso registrado total	$12.748.750
Diferencia total (registrado − esperado)	+$25.000 (0,2 % del esperado)
Filas con discrepancia	1 de 36 (2,8 %): V020, Audífonos, Centro, Distribuidor
Detalle de V020	Registrado $295.000 vs esperado $270.000 (6 × $45.000, sin descuento): sobrefacturación de $25.000, el 9,3 % de esa venta

Producto	Ingreso registrado	% del total	Unidades
Monitor	$4.455.000	34,9 %	45
Mouse	$3.251.250	25,5 %	45
Teclado	$2.992.500	23,5 %	45
Audífonos	$2.050.000	16,1 %	45

Los **tres productos de mayores ingresos** son Monitor, Mouse y Teclado. Los cuatro productos vendieron exactamente 45 unidades, así que la diferencia de ingresos viene del precio neto por unidad (después del descuento): Monitor $99.000, Mouse $72.250, Teclado $66.500 y Audífonos $45.000.

Región	Ingreso	%		Canal	Ingreso	%
Centro	$5.114.500	40,1 %		Distribuidor	$5.114.500	40,1 %
Sur	$4.241.250	33,3 %		Tienda	$3.881.250	30,4 %
Norte	$3.393.000	26,6 %		Web	$3.753.000	29,4 %

Región y canal casi coinciden: Centro = Distribuidor, Sur = Tienda y Norte = Web. La única excepción es V028 (región Sur, canal Web, $360.000), por eso Sur y Tienda difieren en exactamente ese valor. Las cifras de Centro y Distribuidor incluyen los $25.000 de más de V020; sin ese error serían $5.089.500 y el orden no cambia.

Visualizaciones

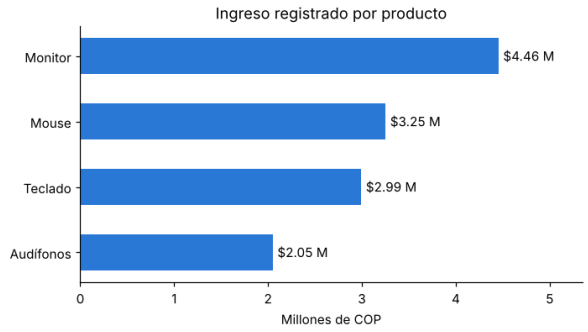


Figura 1. Ingreso por producto.

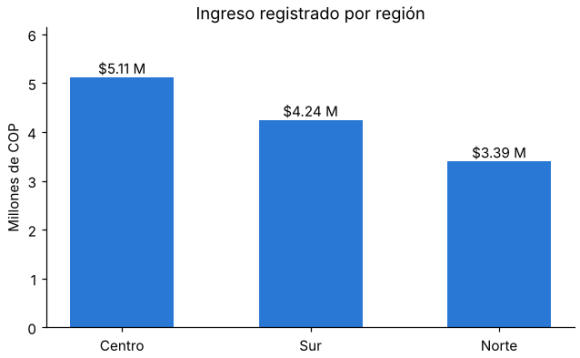


Figura 2. Ingreso por región.

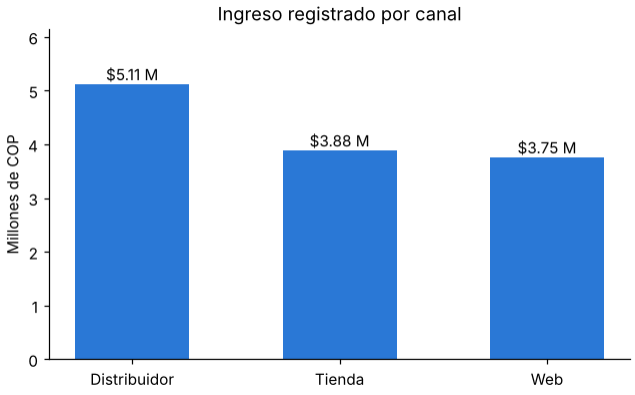


Figura 3. Ingreso por canal.

Decisión gerencial

Priorizar el producto Monitor y el canal Distribuidor.

• Monitor genera el 34,9 % del ingreso ($4.455.000) con las mismas 45 unidades que los demás: es el que más ingreso deja por unidad vendida.

- Distribuidor genera el 40,1 % del ingreso y tiene el ticket más alto: $426.208 por venta (6,0 unidades) frente a $352.841 en Tienda y $288.692 en Web.
- La combinación Monitor–Distribuidor ($1.782.000) es la más alta de las 12 combinaciones producto–canal.
- En calidad de facturación el problema es pequeño pero real: corregir V020 y poner una validación automática de Ingreso_Registrado contra Unidades × Precio × (1 − Descuento).

Ejercicio 2 — VidaPlus: oportunidad del servicio (Power BI)

Enunciado. La dirección de VidaPlus necesita conocer las demoras y la satisfacción de los usuarios. Archivo: 02_VidaPlus_Atenciones.csv (36 atenciones de septiembre de 2026).

E — Extraer

Se importan **36 filas y 7 columnas**: ID_Atencion, Fecha, Servicio, Sede, Espera_Min, Satisfaccion_1a5 y Estado. Hay 3 servicios (Consulta, Laboratorio, Urgencias), 3 sedes (Centro, Norte, Sur) y 2 estados (Atendido, Cancelado).

T — Transformar

#	Transformación	Qué corrige
1	Sede: Recortar + mayúscula cada palabra	A006 «centro␣» → Centro.
2	Espera_Min y Satisfaccion_1a5 → número	A018 trae Espera_Min vacía: queda en blanco (null), no se reemplaza por 0 ni por un promedio.
3	Columna Satisfaccion_Valida: el valor si está entre 1 y 5; si no, null	A026 trae satisfacción 6 (fuera del rango 1–5): se marca como inválida; el dato original se conserva en su columna.
4	Columna Intervalo_Espera: ≤20, 21–40, >40 minutos; «Sin clasificar» si no hay tiempo	A018 queda «Sin clasificar».

Anomalías de este archivo: **3** (A006 sede con formato incorrecto, A018 espera faltante, A026 satisfacción fuera de rango).

L — Cargar

Cerrar y aplicar; la tabla queda como «Atenciones» y se crean las medidas DAX del Anexo A.

Indicadores

Indicador	Resultado
Atenciones totales / atendidas / canceladas	36 / 24 / 12
Esperas con tiempo válido	35 (A018 no tiene tiempo)
% de esperas > 40 min	18 de 35 = **51,4 %**. Denominador: las 35 atenciones con tiempo válido. Si se dividiera entre las 36 atenciones daría 50,0 %, pero A018 no se puede clasificar, por eso no entra.
Distribución por intervalo	≤ 20 min: 5 · 21–40 min: 12 · > 40 min: 18 · Sin clasificar: 1
Satisfacción promedio general	3,00 (35 valores válidos de 36)

Sede	Atenciones	Espera promedio	Tiempos válidos	% espera > 40 min
Centro	13	37,8 min	13	46,2 % (6 de 13)
Norte	11	33,0 min	10	60,0 % (6 de 10)
Sur	12	51,0 min	12	50,0 % (6 de 12)

Servicio	Atenciones	Satisfacción promedio	Valores válidos	Espera promedio
Consulta	12	3,25	12	31,0 min
Laboratorio	12	3,08	12	40,0 min

Servicio	Atenciones	Satisfacción promedio	Valores válidos	Espera promedio
Urgencias	12	2,64	11 (A026 inválida)	51,0 min

Visualizaciones

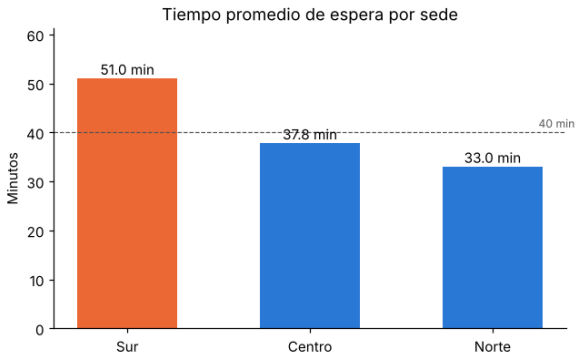


Figura 4. Espera promedio por sede (línea: 40 min).

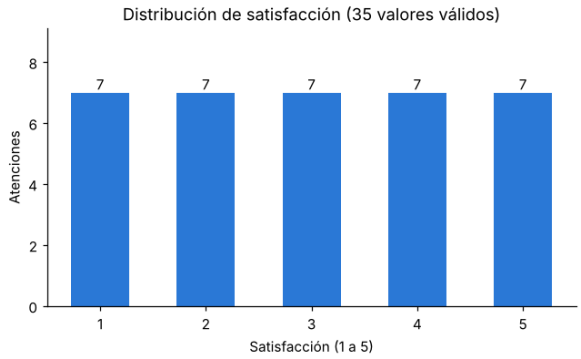


Figura 5. Distribución de satisfacción.

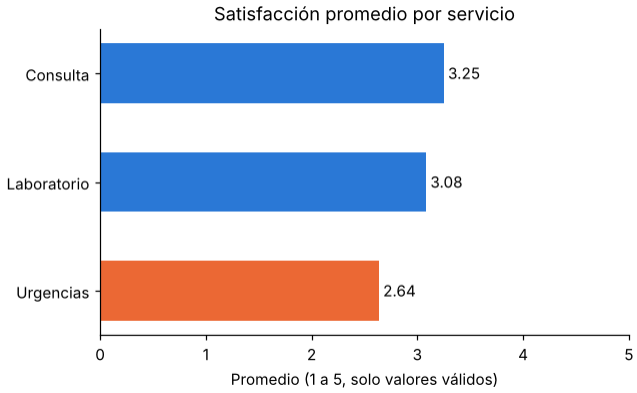


Figura 6. Satisfacción promedio por servicio (solo valores válidos).

Decisión gerencial

Atención prioritaria: servicio de Urgencias en la sede Sur.

• La sede Sur tiene la mayor espera promedio: 51,0 min, 18 min por encima de Norte (33,0) y 13 min por encima de Centro (37,8). Todas sus esperas son de 32 min o más.

• Urgencias tiene la satisfacción más baja (2,64 de 5) frente a 3,25 de Consulta y 3,08 de Laboratorio.

• Las 12 atenciones canceladas son todas de Urgencias en Sur (100 %).

• Limitación: sede, servicio y estado coinciden por completo (Sur = Urgencias = Cancelado), así que con estos datos no se puede separar cuál de los tres factores causa el problema. Además, las atenciones canceladas traen tiempo de espera y calificación, algo raro que habría que revisar en el sistema de origen.

Ejercicio 3 — RutaExpress: retrasos y costos (Python)

Enunciado. La gerencia de RutaExpress busca identificar retrasos por transportista y anomalías de costos. Archivo: 03_RutaExpress_Envios.csv (36 envíos, 3 transportistas, 3 zonas). Código completo en el notebook (Anexo B).

E — Extraer

Se carga el CSV con pandas (encoding utf-8-sig para quitar el BOM del archivo) y se muestran head(), info() e isna(). Resultado: **36 filas y 7 columnas**; el único faltante es Dias_Reales de E030.

```
Filas y columnas: (36, 7)
  ID Envio    Zona Transportista  Dias Planeados  Dias Reales  Costo COP  \
0     E001   Rural            T2               1          1.0      36500
1     E002   Mixta            T3               1          2.0      29500
2     E003  Urbana            T1               1          3.0      22500
3     E004   Rural            T2               1          4.0      15500
4     E005   Mixta            T3               1          1.0      40000

      Estado
0  Entregado
1   Devuelto
2  Entregado
3  Entregado
4   Devuelto

<class 'pandas.DataFrame'>
RangeIndex: 36 entries, 0 to 35
Data columns (total 7 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   ID Envio        36 non-null     str
 1   Zona            36 non-null     str
 2   Transportista   36 non-null     str
 3   Dias Planeados  36 non-null     int64
 4   Dias_Reales     35 non-null     float64
 5   Costo COP       36 non-null     int64
 6   Estado          36 non-null     str
dtypes: float64(1), int64(2), str(4)
memory usage: 2.1 KB

Faltantes por columna (estado original):
ID Envio          0
Zona              0
Transportista     0
Dias Planeados    0
Dias Reales       1
Costo_COP         0
Estado            0
dtype: int64
```

Captura 1 (E3.1). Lectura del CSV original: filas y columnas, head(), info() y faltantes (isna).

T — Transformar

ID	Anomalía	Tratamiento (sin inventar valores)
E010	Zona « urbana » con espacios y minúscula	strip() + title() → «Urbana». Sin esto se contaría como una zona distinta.
E022	Costo_COP = −5.000 (un costo no puede ser negativo)	Se marca en Costo_invalido y el costo pasa a NaN. No se reemplaza por ningún valor.
E030	Dias_Reales vacío	Retraso_Dias queda NaN y se marca en Tiempo_invalido. Sale del cálculo de retrasos (quedan 35 envíos).

Además se convierten Dias_Planeados, Dias_Reales y Costo_COP a número con to_numeric(errors="coerce") y se crea Retraso_Dias = max(Dias_Reales − Dias_Planeados, 0).

```
Zonas antes: [' urbana ', 'Mixta', 'Rural', 'Urbana']
```

```
Zonas después: ['Mixta', 'Rural', 'Urbana']
Costos negativos encontrados: 1
   ID_Envio Transportista   Zona  Costo_COP
21     E022            T2  Rural      -5000
Filas sin tiempo válido: 1
   ID_Envio Transportista  Dias_Planeados  Dias_Reales
29     E030            T1               1          NaN

Faltantes después de limpiar:
Dias Planeados    0
Dias Reales       1
Costo_COP         1
Retraso Dias      1
dtype: int64
```

Captura 2 (E3.2). Limpieza: zonas antes/después, costo negativo, fila sin tiempo y faltantes resultantes.

L — Cargar

Se exporta **RutaExpress_Limpio.csv** (36 filas, 10 columnas: las 7 originales más Costo_invalido, Retraso_Dias y Tiempo_invalido) y se vuelve a leer para comprobar que quedó bien guardado.

```
CSV limpio guardado: (36, 10) (filas, columnas)
  ID_Envio    Zona Transportista  Dias_Planeados  Dias_Reales  Costo_COP  \
0     E001   Rural            T2               1          1.0    36500.0
1     E002   Mixta            T3               1          2.0    29500.0
2     E003  Urbana            T1               1          3.0    22500.0

      Estado  Costo invalido  Retraso Dias  Tiempo invalido
0  Entregado           False           0.0            False
1   Devuelto           False           1.0            False
2  Entregado           False           2.0            False
```

Captura 3 (E3.3). Exportación del CSV limpio y verificación de lectura.

Indicadores

Indicador	Resultado
Envíos con tiempos válidos	35 de 36
Entregas a tiempo / retrasadas	14 / 21
% de retrasos	60,0 % (21 de 35)
Retraso promedio general	1,20 días

Transportista	Envíos válidos	Retrasados	% retraso	Retraso promedio	Estado de sus envíos
T1	11	7	63,6 %	1,27 días	12 Entregado
T2	12	7	58,3 %	1,25 días	12 Entregado
T3	12	7	58,3 %	1,08 días	**12 Devuelto**

Zona	Envíos	Costos válidos	Costo promedio válido	Retraso promedio
Urbana	13	13	$23.577	1,17 días
Rural	11	10 (E022 excluido)	$26.000	1,36 días
Mixta	12	12	$29.500	1,08 días

Anomalías y hallazgos de datos: (1) costo negativo en E022; (2) tiempo real faltante en E030; (3) zona mal escrita en E010; (4) los 12 envíos de T3 figuran como «Devuelto» y ninguno como «Entregado»; (5) Dias_Planeados vale 1 en los 36 envíos, así que cualquier día adicional cuenta como retraso; (6) cada transportista opera casi solo en una zona (T1 Urbana, T2 Rural, T3 Mixta; la única excepción es E010), por lo que transportista y zona están mezclados.

```
Filas con tiempos válidos: 35
Entregas a tiempo: 14  |  Retrasadas: 21
% de retrasos: 60.0%  (denominador = 35 filas válidas)
Retraso promedio general: 1.20 días

               envios_validos  retrasadas  pct_retraso  retraso_prom_dias  \
Transportista
T1                         11           7        63.64               1.27
T2                         12           7        58.33               1.25
T3                         12           7        58.33               1.08

               retraso_max_dias
Transportista
T1                          3.0
T2                          3.0
T3                          3.0

        envios  costos_validos  costo_prom  retraso_prom_dias
Zona
Mixta       12              12    29500.00               1.08
Rural       11              10    26000.00               1.36
Urbana      13              13    23576.92               1.17

Costo promedio por zona SIN limpiar:
Zona
Mixta     29500.00
Rural     23181.82
Urbana    23576.92
Name: Costo COP, dtype: float64

Estado         Devuelto  Entregado
Transportista
T1                    0         12
T2                    0         12
T3                   12          0

Zona           Mixta  Rural  Urbana
Transportista
T1                 0      0      12
T2                 0     11       1
T3                12      0       0
```

Captura 4 (E3.4). Indicadores: entregas a tiempo/retrasadas, % de retraso, resumen por transportista y por zona, costo sin limpiar y tablas cruzadas.

Visualizaciones

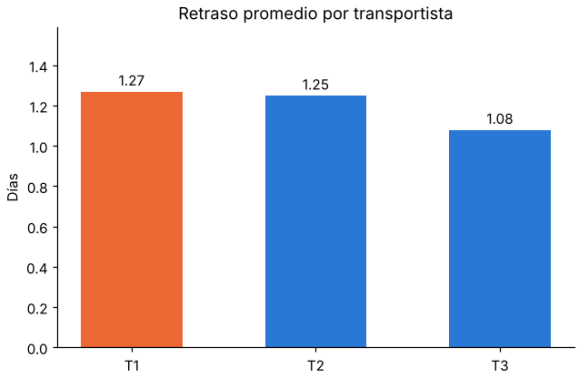


Figura 7. Retraso promedio por transportista.

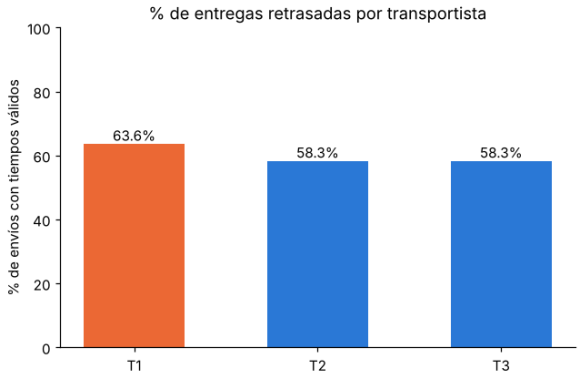


Figura 8. % de entregas retrasadas por transportista.

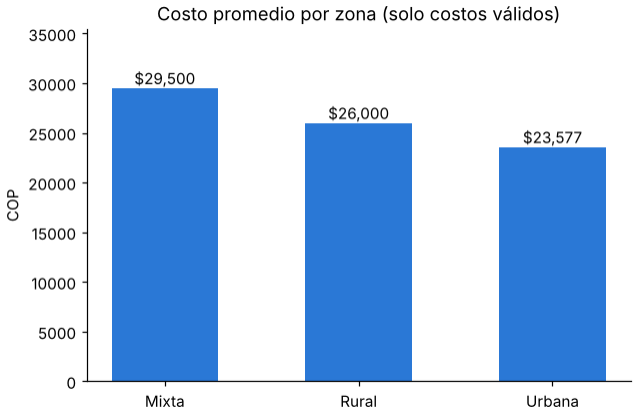


Figura 9. Costo promedio por zona (solo costos válidos).

Decisión gerencial

T1 es el transportista que debe entrar primero a un plan de mejora, y T3 requiere una revisión aparte.

• T1 tiene el mayor % de retraso (63,6 %) y el mayor retraso promedio (1,27 días).

• Pero la diferencia entre T1 y T2 es mínima (1,27 vs 1,25 días; 63,6 % vs 58,3 %): el problema es general, el 60 % de todos los envíos llegó tarde con una promesa de 1 día. El plan debe incluir también a T2 y revisar si el plazo planeado de 1 día es realista.

• T3 tiene el menor retraso (1,08 días), pero sus 12 envíos están en estado «Devuelto» y cobra el costo promedio más alto ($29.500). Eso es más grave que un día de retraso y hay que auditarlo (si es un error de registro o si de verdad devuelve todo).

• Cuidado con el costo: si no se limpiara E022, la zona Rural parecería más barata que la Urbana ($23.182 vs $23.577) cuando en realidad es más cara ($26.000 vs $23.577).

Ejercicio 4 — NovaMarket: clientes y fidelización (Python)

Enunciado. El área comercial requiere reconocer clientes de alto valor y oportunidades de recuperación. Archivo: 04_NovaMarket_Clientes.csv (36 clientes de Bogotá, Medellín y Cali).

E — Extraer

Se importa con pandas: **36 filas y 6 columnas** (ID_Cliente, Segmento, Ciudad, Compras_90d, Gasto_90d_COP, Dias_Ultima_Compra). Hay un faltante en Gasto_90d_COP (C021).

```
Filas y columnas: (36, 6)
  ID_Cliente     Segmento    Ciudad  Compras_90d  Gasto_90d_COP  \
0       C001    Frecuente  Medellín            6       480000.0
1       C002  Corporativo      Cali            3       105000.0
2       C003        Nuevo    Bogotá            8       640000.0
3       C004    Frecuente  Medellín            5       175000.0
4       C005  Corporativo      Cali            2       160000.0

   Dias Ultima Compra
0                  11
1                  22
2                  33
3                  44
4                  55

<class 'pandas.DataFrame'>
RangeIndex: 36 entries, 0 to 35
Data columns (total 6 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   ID Cliente          36 non-null     str
 1   Segmento            36 non-null     str
 2   Ciudad              36 non-null     str
 3   Compras 90d         36 non-null     int64
 4   Gasto 90d COP       35 non-null     float64
 5   Dias_Ultima_Compra  36 non-null     int64
dtypes: float64(1), int64(2), str(3)
memory usage: 1.8 KB

Faltantes por columna (estado original):
ID Cliente            0
Segmento              0
Ciudad                0
Compras 90d           0
Gasto_90d_COP         1
Dias Ultima Compra    0
dtype: int64
```

Captura 5 (E4.1). Lectura del CSV original: filas y columnas, head(), info() y faltantes.

T — Transformar

ID	Anomalía	Tratamiento
C011	Ciudad « bogotá » con espacio y minúscula	strip() + title() → «Bogotá».
C021	Gasto_90d_COP vacío	Queda NaN y marcado en Gasto_faltante. Al faltar un dato esencial → No clasificable.
C032	Compras_90d = −1	Se marca en Compras_invalida y pasa a NaN. → No clasificable.

Regla de clasificación (en este orden): si falta compras, gasto o días de última compra → *No clasificable*; si gasto ≥ $250.000 y compras ≥ 3 → *Alto valor*; si no, y la última compra fue hace más de 60 días → *En seguimiento*; en los demás casos → *Otros*. Se agregó la columna Alto_valor_en_riesgo (Alto valor con más de 60 días sin comprar) porque la regla de la guía solo mira «En seguimiento» cuando el cliente no es de alto valor.

```
Ciudades antes: ['Bogotá', 'Cali', 'Medellín', 'bogotá ']
Ciudades después: ['Bogotá', 'Cali', 'Medellín']
Compras negativas: 1 | Gastos faltantes: 1
   ID Cliente     Segmento  Compras 90d  Gasto 90d COP  Dias Ultima Compra
20       C021        Nuevo            2            NaN                  49
31       C032  Corporativo           -1        35000.0                  79

Categoria
Otros              15
Alto valor         14
En seguimiento      5
No clasificable     2
Name: count, dtype: int64
```

Captura 6 (E4.2). Limpieza y clasificación: ciudades antes/después, valores inválidos y conteo por categoría.

L — Cargar

Se exporta **NovaMarket_Clientes_Limpio.csv** (36 filas, 10 columnas: las 6 originales más Compras_invalida, Gasto_faltante, Categoria y Alto_valor_en_riesgo) y se verifica leyéndolo de nuevo.

```
CSV depurado guardado: (36, 10) (filas, columnas)
  ID Cliente     Segmento    Ciudad  Compras 90d  Gasto 90d COP  \
0       C001    Frecuente  Medellín          6.0       480000.0
1       C002  Corporativo      Cali          3.0       105000.0
2       C003        Nuevo    Bogotá          8.0       640000.0

   Dias_Ultima_Compra  Compras_invalida  Gasto_faltante   Categoria  \
0                  11             False           False  Alto valor
1                  22             False           False       Otros
2                  33             False           False  Alto valor

   Alto valor en riesgo
0                 False
1                 False
2                 False
```

Captura 7 (E4.3). Exportación del CSV depurado y verificación de lectura.

Indicadores

Categoría	Clientes	% de clientes	Gasto total	Gasto promedio
Alto valor	14	38,9 %	$6.880.000	$491.429
Otros	15	41,7 %	$2.370.000	$158.000
En seguimiento	5	13,9 %	$595.000	$119.000
No clasificable	2	5,6 %	$35.000	$35.000 (solo C032)
Total	**36**	100 %	**$9.880.000**	**$282.286** (35 clientes con gasto válido)

Los clientes de Alto valor son el 38,9 % de la base pero concentran el **69,6 %** del gasto. El gasto total y el promedio usan solo gastos válidos (35 clientes); C021 no tiene gasto.

Top 5 por gasto	Segmento	Ciudad	Compras	Gasto	Días sin comprar
C003	Nuevo	Bogotá	8	$640.000	33
C011	Corporativo	Bogotá	8	$640.000	30
C019	Frecuente	Medellín	8	$640.000	27
C027	Nuevo	Bogotá	8	$640.000	24
C035	Corporativo	Cali	8	$640.000	21

Hay cinco clientes empatados en el gasto más alto ($640.000), por eso el top 5 es exactamente ese grupo; juntos suman $3.200.000 (32,4 % del gasto).

En seguimiento	Segmento	Ciudad	Compras	Gasto	Días sin comprar
C006	Nuevo	Bogotá	7	$245.000	66
C014	Corporativo	Cali	7	$245.000	63
C008	Corporativo	Cali	1	$35.000	88
C016	Frecuente	Medellín	1	$35.000	85
C024	Nuevo	Bogotá	1	$35.000	82

Alto valor en riesgo (> 60 días)	Segmento	Ciudad	Compras	Gasto	Días sin comprar
C033	Nuevo	Bogotá	6	$480.000	90
C007	Frecuente	Medellín	4	$320.000	77
C015	Nuevo	Bogotá	4	$320.000	74
C023	Corporativo	Cali	4	$320.000	71
C031	Frecuente	Medellín	4	$320.000	68

Estos 5 clientes de Alto valor llevan más de 60 días sin comprar y suman $1.760.000 (17,8 % del gasto total). No salen en «En seguimiento» porque la regla da prioridad a Alto valor.

```
                clientes  gasto total  gasto prom
Categoria
Alto valor            14    6880000.0    491429.0
En seguimiento         5     595000.0    119000.0
No clasificable        2      35000.0     35000.0
Otros                 15    2370000.0    158000.0

Gasto total (todos los clientes con gasto válido): $9,880,000
Gasto promedio válido: $282,286 (sobre 35 clientes con gasto)

Top 5 por gasto:
   ID Cliente     Segmento    Ciudad  Compras 90d  Gasto 90d COP  \
2        C003        Nuevo    Bogotá          8.0       640000.0
18       C019    Frecuente  Medellín          8.0       640000.0
10       C011  Corporativo    Bogotá          8.0       640000.0
34       C035  Corporativo      Cali          8.0       640000.0
26       C027        Nuevo    Bogotá          8.0       640000.0

    Dias Ultima Compra
2                   33
18                  27
10                  30
34                  21
26                  24

Clientes En seguimiento:
   ID Cliente     Segmento    Ciudad  Compras 90d  Gasto 90d COP  \
5        C006        Nuevo    Bogotá          7.0       245000.0
7        C008  Corporativo      Cali          1.0        35000.0
13       C014  Corporativo      Cali          7.0       245000.0
15       C016    Frecuente  Medellín          1.0        35000.0
23       C024        Nuevo    Bogotá          1.0        35000.0

    Dias Ultima Compra
5                   66
7                   88
13                  63
15                  85
23                  82

Alto valor en riesgo (>60 días sin comprar):
   ID Cliente     Segmento    Ciudad  Compras 90d  Gasto 90d COP  \
6        C007    Frecuente  Medellín          4.0       320000.0
14       C015        Nuevo    Bogotá          4.0       320000.0
```

```
22       C023  Corporativo      Cali          4.0       320000.0
... (salida recortada; está completa en el notebook)
```

Captura 8 (E4.4). Indicadores: clientes por categoría, gasto total y promedio, top 5, en seguimiento, alto valor en riesgo y no clasificables.

Visualizaciones

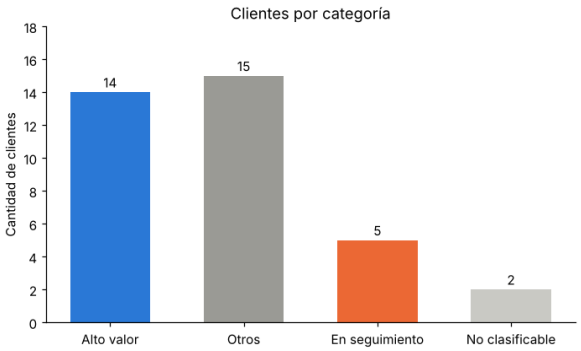


Figura 10. Clientes por categoría.

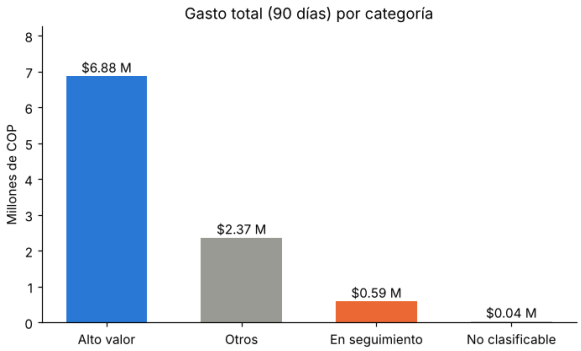


Figura 11. Gasto total por categoría.

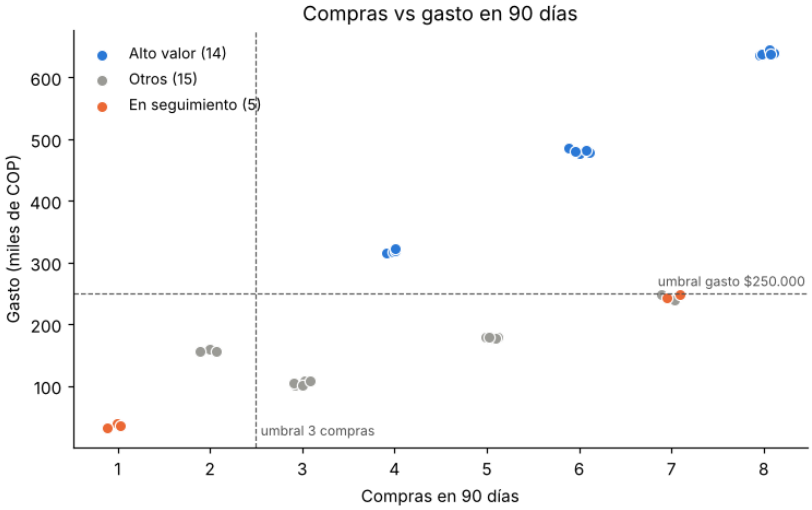


Figura 12. Compras vs gasto en 90 días (34 clientes; C021 y C032 no se pueden graficar). Los puntos están movidos un poco solo para que no se tapen.

Decisión gerencial

Priorizar la campaña en dos grupos, en este orden.

• **Recuperación inmediata:** los 5 clientes de Alto valor en riesgo (C033, C007, C015, C023, C031): $1.760.000 de gasto y entre 68 y 90 días sin comprar. Son clientes que ya demostraron que compran 4 a 6 veces y gastan $320.000 a $480.000 en 90 días.

• **Fidelización:** los 5 clientes top (C003, C011, C019, C027, C035), con 8 compras y $640.000 cada uno, el 32,4 % del gasto, todos con compra reciente (21 a 33 días).

• **Oportunidad:** C006 y C014 están a solo $5.000 del umbral de Alto valor (7 compras y $245.000) y llevan 63 a 66 días sin comprar.

• Los otros tres «En seguimiento» (C008, C016, C024) compraron una vez y gastaron $35.000: solo justifican una reactivación de bajo costo (por ejemplo, correo o mensaje digital).

- Antes de la campaña hay que corregir C021 y C032 para poder clasificarlos.

Tabla comparativa de hallazgos y calidad de datos

Comparación de las tres organizaciones. Cada archivo trajo exactamente 3 anomalías (12 en total; 8,3 % de los 144 registros).

Aspecto	NovaMarket Ventas	VidaPlus	RutaExpress	NovaMarket Clientes
Herramienta	Power BI	Power BI	Python	Python
Registros / columnas	36 / 9	36 / 7	36 / 7	36 / 6
Anomalías	3 (V009, V028, V020)	3 (A006, A018, A026)	3 (E010, E022, E030)	3 (C011, C021, C032)
Texto con espacios/mayúsculas	2 (Region, Canal)	1 (Sede)	1 (Zona)	1 (Ciudad)
Valores faltantes	0	1 (Espera_Min)	1 (Dias_Reales)	1 (Gasto_90d_COP)
Valores inválidos o fuera de rango	0	1 (satisfacción = 6)	1 (costo = −5.000)	1 (compras = −1)
Inconsistencia de cálculo	1 (V020: +$25.000)	0	0	0
Filas afectadas	3 (8,3 %)	3 (8,3 %)	3 (8,3 %)	3 (8,3 %)
Filas eliminadas	0	0	0	0
Calidad después de limpiar	Alta: 35 de 36 filas sin discrepancia	Buena: 35 tiempos y 35 satisfacciones válidos	Buena: 35 tiempos y 35 costos válidos	Buena: 34 clientes clasificables de 36
Hallazgo principal	Monitor (34,9 %) y Distribuidor (40,1 %) lideran el ingreso	Sur/Urgencias: 51,0 min de espera y satisfacción 2,64	60 % de envíos con retraso; T3 con 100 % devueltos	Alto valor = 38,9 % de clientes y 69,6 % del gasto
Riesgo de decisión si no se limpia	Categorías duplicadas («web ␣», «␣norte␣»)	Satisfacción de Urgencias subiría de 2,64 a 2,92	Rural parecería más barata que Urbana	Clientes con datos malos entrarían a campañas

En general los datos están en buen estado: pocas filas afectadas y todas con tratamiento claro. La diferencia entre las empresas está en el tipo de problema. NovaMarket Ventas tiene el único error «de negocio» (facturación); VidaPlus y RutaExpress tienen datos faltantes o fuera de rango que afectan promedios de servicio; y en Clientes el problema impide clasificar a 2 personas.

Tres respuestas sobre el proceso ETL

EXTRACT — ¿Qué riesgos para la confiabilidad pueden aparecer al extraer datos de diferentes fuentes y cómo verificar su integridad?

Al juntar fuentes distintas puede pasar que cada una escriba los datos a su manera (mayúsculas, espacios, formatos de fecha o de número), que lleguen incompletas, que cambie la codificación del archivo o que haya archivos duplicados o desactualizados. Si eso no se detecta al inicio, los errores viajan hasta el tablero sin que nadie los note.

Casos reales de este laboratorio: (1) los cuatro CSV traen un carácter BOM al inicio que puede dañar el nombre de la primera columna si no se usa la codificación correcta (utf-8-sig en Python; UTF-8 en Power Query); (2) las fuentes mezclan formatos de texto: «␣norte␣», «web␣», «centro␣», «␣urbana␣», «bogotá␣»; (3) hay vacíos en Espera_Min, Dias_Reales y Gasto_90d_COP; (4) el zip traía un archivo duplicado («02_VidaPlus_Atenciones Copiar.csv») que, de haberse cargado como otro dato, habría duplicado las 36 atenciones.

Cómo verificar la integridad: contar filas y columnas contra lo esperado (36 por archivo, 144 en total); revisar tipos con info() o en Power Query; contar faltantes con isna(); comparar archivos sospechosos de duplicado (en este caso un diff mostró que eran idénticos); revisar los valores únicos de las columnas categóricas; y dejar los originales intactos como copia de referencia.

TRANSFORM — ¿Por qué es imprescindible depurar y validar antes de calcular indicadores? Ilustre un caso del laboratorio donde una anomalía alteraría una decisión.

Porque un indicador es solo tan bueno como los datos que lo alimentan: un valor imposible o faltante entra al promedio igual que uno correcto y lo distorsiona sin avisar. Depurar significa decidir con un criterio explícito qué se corrige, qué se marca y qué se excluye, sin inventar valores.

Caso principal (RutaExpress, E022): el envío E022 tiene un costo de −$5.000. Si se deja así, el costo promedio de la zona Rural queda en $23.182 y parece más barato que el de la Urbana ($23.577). Al marcar el costo negativo como inválido, Rural pasa a $26.000, que es $2.423 más caro que la Urbana. Con el dato sucio, la gerencia podría concluir que los envíos rurales son los más baratos de operar, y con el dato limpio ocurre lo contrario.

Otros casos: (a) VidaPlus A026 trae satisfacción 6 en una escala de 1 a 5; si se incluyera, la satisfacción de Urgencias subiría de 2,64 a 2,92 y el problema del servicio se vería menor de lo que es. (b) VidaPlus A018 no tiene tiempo de espera; si se tratara como 0, la espera promedio de Norte bajaría de 33,0 a 30,0 minutos. (c) NovaMarket V028 tiene el canal «web␣»; sin recortar el texto, Power BI mostraría una categoría aparte y el canal Web aparecería con $360.000 menos. (d) NovaMarket V020 está sobrefacturada en $25.000; solo se descubre porque se compara el ingreso registrado contra el esperado.

LOAD — ¿Qué diferencia hay entre cargar datos originales y datos preparados? ¿Cómo conserva la trazabilidad de los cambios?

Los datos originales son la copia tal como llegó de la fuente, con todas sus anomalías; sirven de evidencia y no se tocan. Los datos preparados son el resultado de aplicar las transformaciones y son los que alimentan los indicadores y gráficos. Cargar los originales produce cifras que no son confiables; cargar solo los preparados sin documentar produce cifras que nadie puede comprobar.

La trazabilidad se conserva así: (1) los CSV originales quedan intactos y los preparados se guardan como archivos nuevos (RutaExpress_Limpio.csv y NovaMarket_Clientes_Limpio.csv); (2) en Power

BI los «Pasos aplicados» de Power Query dejan registrado cada paso en orden; (3) en Python, el notebook contiene cada transformación con su comentario y se puede ejecutar de nuevo desde el original; (4) las anomalías se marcan en columnas propias (Costo_invalido, Tiempo_invalido, Compras_invalida, Gasto_faltante) en vez de borrar filas: el CSV limpio tiene las mismas 36 filas que el original; (5) la bitácora del Anexo C lista cada cambio con el valor original, el tratamiento y el resultado.

Conclusiones y recomendaciones

Conclusiones

1. **Los datos tenían problemas pequeños pero con efecto real.** Cada archivo traía 3 anomalías (12 de 144 registros, 8,3 %). Todas se pudieron tratar sin borrar filas ni inventar valores, y varias cambiaban una conclusión: por ejemplo, el costo negativo de E022 invierte la comparación Rural–Urbana y la satisfacción 6 de A026 oculta parte del problema de Urgencias.
2. **En NovaMarket el ingreso está concentrado.** Monitor aporta el 34,9 % de las ventas y el canal Distribuidor el 40,1 %; la facturación es casi exacta (una sola discrepancia de $25.000, 0,2 %). En clientes, el 38,9 % de la base (Alto valor) genera el 69,6 % del gasto y 5 de esos clientes llevan más de 60 días sin comprar.
3. **VidaPlus y RutaExpress tienen problemas de servicio que no dependen de un solo punto.** En VidaPlus la sede Sur/Urgencias tiene 51,0 min de espera, satisfacción 2,64 y todas las cancelaciones. En RutaExpress el 60 % de los envíos llega tarde en los tres transportistas (T1 es el peor por muy poco) y T3 tiene el 100 % de sus envíos devueltos.

Recomendaciones accionables

1. **Validar los datos en la fuente.** Usar listas desplegables para región, canal, sede, zona y ciudad; limitar la satisfacción a 1–5; no permitir costos ni compras negativos; hacer obligatorios los tiempos y el gasto; y calcular automáticamente el ingreso en facturación (Unidades × Precio × (1 − Descuento)) para que no se repita un error como el de V020 (corregirlo ahora).
2. **Plan de mejora de servicio con metas medibles.** VidaPlus: revisar la operación de Urgencias en Sur (turnos, triage, causas de cancelación) con la meta de bajar la espera promedio de 51 a menos de 40 minutos y medir de nuevo la satisfacción. RutaExpress: acordar un plan con T1 (mayor retraso), auditar las devoluciones de T3 y revisar si el plazo planeado de 1 día es realista para cada zona.
3. **Campaña de clientes en dos fases.** Primero recuperar a los 5 clientes de Alto valor en riesgo (C033, C007, C015, C023, C031) con una oferta personalizada, y luego fidelizar a los 5 del top ($640.000 cada uno) y empujar a C006 y C014, que están a $5.000 de ser Alto valor. Medir el resultado con el gasto a 90 días y los días desde la última compra.

Fuentes de información y bibliografía

Universidad EAN. (2026). Datos_ETL_Cuatro_Desafios.zip y guía de la Learning Activity 3 – Data Intelligence Challenge [Material de clase]. Bogotá, Colombia.

Microsoft. (s. f.). Power Query documentation. Microsoft Learn. https://learn.microsoft.com/en-us/power-query/

Microsoft. (s. f.). DAX overview. Microsoft Learn. https://learn.microsoft.com/en-us/dax/dax-overview

The pandas development team. (s. f.). pandas documentation. https://pandas.pydata.org/docs/

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95.

Thompson, J. K. (citado en la guía de la actividad): «Data becomes knowledge when it is analyzed, and knowledge creates impact when it is used securely and responsibly.»

Anexos

Contenido de los anexos: A) código de Power Query y medidas DAX para los ejercicios 1 y 2; B) código Python de los ejercicios 3 y 4; C) bitácora de cambios; D) archivos depurados.

Anexo A. Código de Power Query (M) y medidas DAX

Ejercicio 1 — Power Query (Editor avanzado)

```
let
    Origen = Csv.Document(File.Contents("C:\Ruta\01 NovaMarket Ventas.csv"), [Delimiter=",",
Encoding=65001]),
    Encabezados = Table.PromoteHeaders(Origen, [PromoteAllScalars=true]),
    Tipos = Table.TransformColumnTypes(Encabezados, {{"Fecha", type date}, {"Unidades", Int64.Type},
        {"Precio Unitario", Int64.Type}, {"Descuento Pct", Int64.Type}, {"Ingreso Registrado", type
number}}, "en-US"),
    Texto = Table.TransformColumns(Tipos, {{"Region", each Text.Proper(Text.Trim( )), type text},
        {"Canal", each Text.Proper(Text.Trim( )), type text}}),
    Esperado = Table.AddColumn(Texto, "Ingreso Esperado",
        each [Unidades] * [Precio Unitario] * (1 - [Descuento Pct] / 100), type number),
    Diferencia = Table.AddColumn(Esperado, "Diferencia", each [Ingreso Registrado] -
[Ingreso Esperado], type number)
in
    Diferencia
```

Ejercicio 1 — Medidas DAX (tabla Ventas)

```
Ingreso Esperado Total = SUM(Ventas[Ingreso Esperado])
Ingreso Registrado Total = SUM(Ventas[Ingreso Registrado])
Diferencia Total = [Ingreso Registrado Total] - [Ingreso Esperado Total]
Filas con Discrepancia = CALCULATE(COUNTROWS(Ventas), FILTER(Ventas, ABS(Ventas[Diferencia]) > 0.01))
Cuantia Discrepancias = SUMX(Ventas, ABS(Ventas[Diferencia]))
```

Ejercicio 2 — Power Query (Editor avanzado)

```
let
    Origen = Csv.Document(File.Contents("C:\Ruta\02 VidaPlus Atenciones.csv"), [Delimiter=",",
Encoding=65001]),
    Encabezados = Table.PromoteHeaders(Origen, [PromoteAllScalars=true]),
    Texto = Table.TransformColumns(Encabezados, {{"Sede", each Text.Proper(Text.Trim( )), type text},
        {"Servicio", each Text.Trim( ), type text}}),
    Tipos = Table.TransformColumnTypes(Texto, {{"Fecha", type date}, {"Espera Min", type number},
        {"Satisfaccion 1a5", type number}}, "en-US"),
    SatValida = Table.AddColumn(Tipos, "Satisfaccion Valida",
        each if [Satisfaccion 1a5] <> null and [Satisfaccion 1a5] >= 1 and [Satisfaccion 1a5] <= 5
             then [Satisfaccion 1a5] else null, type number),
    Intervalo = Table.AddColumn(SatValida, "Intervalo Espera",
        each if [Espera Min] = null then "Sin clasificar"
             else if [Espera Min] <= 20 then "<=20 min"
             else if [Espera Min] <= 40 then "21-40 min" else ">40 min", type text)
in
    Intervalo
```

Ejercicio 2 — Medidas DAX (tabla Atenciones)

```
Atenciones Totales = COUNTROWS(Atenciones)
Atendidas = CALCULATE(COUNTROWS(Atenciones), Atenciones[Estado] = "Atendido")
Canceladas = CALCULATE(COUNTROWS(Atenciones), Atenciones[Estado] = "Cancelado")
Espera Promedio = AVERAGE(Atenciones[Espera Min])          -- ignora los vacíos
Esperas Validas = COUNT(Atenciones[Espera Min])            -- denominador del % (35)
Esperas Mayor 40 = CALCULATE(COUNTROWS(Atenciones), Atenciones[Espera Min] > 40)
Pct Espera Mayor 40 = DIVIDE([Esperas Mayor 40], [Esperas Validas])
Satisfaccion Promedio = AVERAGE(Atenciones[Satisfaccion_Valida])
```

Cambiar «C:\Ruta\» por la carpeta real donde están los CSV. Los pasos también se pueden hacer con la interfaz; el código sirve de respaldo.

Anexo B. Código Python (notebook)

Archivo: Notebook_Ej3_Ej4.ipynb. A continuación el código de las celdas, con los comentarios que explican cada paso.

Configuración

```
# Librerías que se usan en todo el notebook
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os

os.makedirs("graficos", exist_ok=True)   # carpeta donde guardo los gráficos para el informe

# Colores que uso en los gráficos (azul = normal, naranja = lo que hay que mirar)
AZUL, NARANJA, GRIS = "#2a78d6", "#eb6834", "#9a9a95"
plt.rcParams.update({"axes.spines.top": False, "axes.spines.right": False, "figure.dpi": 110})
```

Ejercicio 3 — Extraer

```
# Cargo el CSV original. encoding="utf-8-sig" quita el BOM que trae el archivo
# (sin esto la primera columna puede quedar con un carácter invisible al inicio del nombre)
df = pd.read_csv("03 RutaExpress Envios.csv", encoding="utf-8-sig")

print("Filas y columnas:", df.shape)   # esperado: 36 filas, 7 columnas
print(df.head())                       # primeras filas, estado original
print()
df.info()                              # tipos de dato por columna
print()
print("Faltantes por columna (estado original):")
print(df.isna().sum())                 # aquí sale 1 faltante en Dias_Reales (E030)
```

Ejercicio 3 — Transformar

```
# Guardo una copia del original para poder comparar antes/después (trazabilidad)
original = df.copy()

# 1) Zona: quito espacios y dejo formato Título. En E010 venía " urbana " (espacios + minúscula),
#    sin esto Python lo contaría como una zona distinta de "Urbana".
print("Zonas antes:", sorted(df["Zona"].unique()))
df["Zona"] = df["Zona"].str.strip().str.title()
print("Zonas después:", sorted(df["Zona"].unique()))

# 2) Columnas numéricas: errors="coerce" convierte lo que no sea número en NaN (no inventa valores)
for c in ["Dias Planeados", "Dias Reales", "Costo COP"]:
    df[c] = pd.to_numeric(df[c], errors="coerce")

# 3) Costo negativo: un costo de envío no puede ser negativo (E022 = -5000).
#    Lo marco en una columna aparte para documentarlo y lo paso a NaN (no lo reemplazo por ningún valor).
df["Costo invalido"] = df["Costo COP"] < 0
print("Costos negativos encontrados:", int(df["Costo invalido"].sum()))
print(df.loc[df["Costo_invalido"], ["ID_Envio", "Transportista", "Zona", "Costo_COP"]])
df.loc[df["Costo COP"] < 0, "Costo COP"] = np.nan

# 4) Retraso en días = días reales - planeados, pero nunca menos de 0 (llegar antes no es retraso).
#    Si falta alguno de los dos tiempos, el resultado queda NaN (E030 no tiene Dias Reales).
df["Retraso_Dias"] = (df["Dias_Reales"] - df["Dias_Planeados"]).clip(lower=0)
df["Tiempo_invalido"] = df["Dias_Reales"].isna() | df["Dias_Planeados"].isna()
print("Filas sin tiempo válido:", int(df["Tiempo invalido"].sum()))
print(df.loc[df["Tiempo invalido"], ["ID Envio", "Transportista", "Dias Planeados", "Dias Reales"]])

print()
print("Faltantes después de limpiar:")
print(df[["Dias_Planeados", "Dias_Reales", "Costo_COP", "Retraso_Dias"]].isna().sum())
```

Ejercicio 3 — Cargar

```
# El CSV limpio es un archivo nuevo; el original queda intacto.
df.to_csv("RutaExpress Limpio.csv", index=False)

# Compruebo que se guardó bien volviendo a leerlo
```

```
chequeo = pd.read csv("RutaExpress Limpio.csv")
print("CSV limpio guardado:", chequeo.shape, "(filas, columnas)")
print(chequeo.head(3))
```

Ejercicio 3 — Indicadores

```
# Solo se evalúan las filas con tiempos válidos (35 de 36)
v = df[~df["Tiempo invalido"]].copy()
v["Retrasado"] = v["Retraso_Dias"] > 0

a tiempo = int((~v["Retrasado"]).sum())
retrasadas = int(v["Retrasado"].sum())
print(f"Filas con tiempos válidos: {len(v)}")
print(f"Entregas a tiempo: {a tiempo}  |  Retrasadas: {retrasadas}")
print(f"% de retrasos: {retrasadas / len(v) * 100:.1f}%  (denominador = {len(v)} filas válidas)")
print(f"Retraso promedio general: {v['Retraso Dias'].mean():.2f} días")

# Por transportista: cantidad, retrasadas, % retraso, retraso promedio
por transp = v.groupby("Transportista").agg(
    envios validos=("ID Envio", "count"),
    retrasadas=("Retrasado", "sum"),
    pct retraso=("Retrasado", lambda s: s.mean() * 100),
    retraso prom dias=("Retraso Dias", "mean"),
    retraso_max_dias=("Retraso_Dias", "max"),
).round(2)
print()
print(por_transp)

# Costo promedio válido por zona (el costo negativo ya es NaN y pandas lo ignora en el promedio)
por_zona = df.groupby("Zona").agg(
    envios=("ID Envio", "count"),
    costos validos=("Costo COP", "count"),
    costo_prom=("Costo_COP", "mean"),
    retraso prom dias=("Retraso Dias", "mean"),
).round(2)
print()
print(por zona)

# Qué hubiera pasado si NO se hubiera limpiado el costo negativo (para justificar la limpieza)
sin limpiar = original.assign(Costo COP=pd.to numeric(original["Costo COP"], errors="coerce"))
sin limpiar["Zona"] = sin limpiar["Zona"].str.strip().str.title()
print()
print("Costo promedio por zona SIN limpiar:")
print(sin limpiar.groupby("Zona")["Costo COP"].mean().round(2))

# Tablas cruzadas para entender las anomalías
print()
print(pd.crosstab(df["Transportista"], df["Estado"]))
print()
print(pd.crosstab(df["Transportista"], df["Zona"]))
```

Ejercicio 3 — Gráficos

```
# Gráfico 1: retraso promedio por transportista (el naranja marca el más alto)
vals = por transp["retraso prom dias"]
colores = [NARANJA if x == vals.max() else AZUL for x in vals]
fig, ax = plt.subplots(figsize=(6, 4))
barras = ax.bar(vals.index, vals.values, color=colores, width=0.55)
for b, x in zip(barras, vals.values):
    ax.text(b.get_x() + b.get_width() / 2, x + 0.03, f"{x:.2f}", ha="center", fontsize=10)
ax.set title("Retraso promedio por transportista")
ax.set ylabel("Días"); ax.set ylim(0, vals.max() * 1.25)
plt.tight_layout(); plt.savefig("graficos/g3_retraso_transportista.png"); plt.show()

# Gráfico 2: % de entregas retrasadas por transportista
pct = por_transp["pct_retraso"]
colores = [NARANJA if x == pct.max() else AZUL for x in pct]
fig, ax = plt.subplots(figsize=(6, 4))
barras = ax.bar(pct.index, pct.values, color=colores, width=0.55)
for b, x in zip(barras, pct.values):
    ax.text(b.get x() + b.get width() / 2, x + 1, f"{x:.1f}%", ha="center", fontsize=10)
ax.set_title("% de entregas retrasadas por transportista")
ax.set ylabel("% de envíos con tiempos válidos"); ax.set ylim(0, 100)
plt.tight layout(); plt.savefig("graficos/g3 pct retraso transportista.png"); plt.show()
```

```
# Gráfico 3: costo promedio válido por zona
cz = por zona["costo prom"]
fig, ax = plt.subplots(figsize=(6, 4))
barras = ax.bar(cz.index, cz.values, color=AZUL, width=0.55)
for b, x in zip(barras, cz.values):
    ax.text(b.get_x() + b.get_width() / 2, x + 400, f"${x:,.0f}", ha="center", fontsize=10)
ax.set title("Costo promedio por zona (solo costos válidos)")
ax.set ylabel("COP"); ax.set ylim(0, cz.max() * 1.2)
plt.tight_layout(); plt.savefig("graficos/g3_costo_zona.png"); plt.show()
```

Ejercicio 4 — Extraer

```
cl = pd.read csv("04 NovaMarket Clientes.csv", encoding="utf-8-sig")

print("Filas y columnas:", cl.shape)   # esperado: 36 filas, 6 columnas
print(cl.head())
print()
cl.info()
print()
print("Faltantes por columna (estado original):")
print(cl.isna().sum())                 # 1 faltante en Gasto_90d_COP (C021)
```

Ejercicio 4 — Transformar

```
cl original = cl.copy()   # copia para comparar antes/después

# 1) Ciudad: quito espacios y dejo formato Título. En C011 venía "bogotá " (minúscula + espacio).
print("Ciudades antes:", sorted(cl["Ciudad"].unique()))
cl["Ciudad"] = cl["Ciudad"].str.strip().str.title()
print("Ciudades después:", sorted(cl["Ciudad"].unique()))

# 2) Convierto a número lo que debe ser número (lo que no se pueda queda NaN)
for c in ["Compras_90d", "Gasto_90d_COP", "Dias_Ultima_Compra"]:
    cl[c] = pd.to numeric(cl[c], errors="coerce")

# 3) Compras negativas: no puede haber -1 compras (C032). Lo marco y lo paso a NaN, sin inventar un valor.
cl["Compras invalida"] = cl["Compras 90d"] < 0
cl["Gasto faltante"] = cl["Gasto 90d COP"].isna()
print("Compras negativas:", int(cl["Compras_invalida"].sum()), "| Gastos faltantes:",
int(cl["Gasto faltante"].sum()))
print(cl.loc[cl["Compras invalida"] | cl["Gasto faltante"],
             ["ID_Cliente", "Segmento", "Compras_90d", "Gasto_90d_COP", "Dias_Ultima_Compra"]])
cl.loc[cl["Compras 90d"] < 0, "Compras 90d"] = np.nan

# 4) Clasificación según las reglas de la guía. El orden de las reglas importa:
#    primero se revisa si faltan datos, luego Alto valor, luego En seguimiento.
def clasificar(r):
    if pd.isna(r["Compras_90d"]) or pd.isna(r["Gasto_90d_COP"]) or pd.isna(r["Dias_Ultima_Compra"]):
        return "No clasificable"
    if r["Gasto 90d COP"] >= 250000 and r["Compras 90d"] >= 3:
        return "Alto valor"
    if r["Dias Ultima Compra"] > 60:
        return "En seguimiento"
    return "Otros"

cl["Categoria"] = cl.apply(clasificar, axis=1)

# Columna extra para el análisis: clientes de Alto valor que llevan más de 60 días sin comprar (riesgo de fuga)
cl["Alto valor en riesgo"] = (cl["Categoria"] == "Alto valor") & (cl["Dias Ultima Compra"] > 60)
print()
print(cl["Categoria"].value_counts())
```

Ejercicio 4 — Cargar

```
cl.to csv("NovaMarket Clientes Limpio.csv", index=False)

chequeo = pd.read csv("NovaMarket Clientes Limpio.csv")
print("CSV depurado guardado:", chequeo.shape, "(filas, columnas)")
print(chequeo.head(3))
```

Ejercicio 4 — Indicadores

```
# Clientes por categoría + gasto total y promedio (el promedio usa solo gastos válidos)
resumen = cl.groupby("Categoria").agg(
    clientes=("ID_Cliente", "count"),
    gasto total=("Gasto 90d COP", "sum"),
    gasto prom=("Gasto 90d COP", "mean"),
).round(0)
print(resumen)
print()
print("Gasto total (todos los clientes con gasto válido):", f"${cl['Gasto_90d_COP'].sum():,.0f}")
print("Gasto promedio válido:", f"${cl['Gasto 90d COP'].mean():,.0f}", f"(sobre {cl['Gasto 90d COP'].count()}
clientes con gasto)")

# Top 5 por gasto. OJO: hay 5 clientes empatados en 640.000, así que el top 5 son exactamente esos
top5 = cl.sort values("Gasto 90d COP", ascending=False).head(5)
print()
print("Top 5 por gasto:")
print(top5[["ID Cliente", "Segmento", "Ciudad", "Compras 90d", "Gasto 90d COP", "Dias Ultima Compra"]])

# Clientes En seguimiento
print()
print("Clientes En seguimiento:")
print(cl[cl["Categoria"] == "En seguimiento"][["ID Cliente", "Segmento", "Ciudad", "Compras 90d",
"Gasto 90d COP", "Dias Ultima Compra"]])

# Alto valor que llevan más de 60 días sin comprar: son la prioridad de recuperación
print()
print("Alto valor en riesgo (>60 días sin comprar):")
print(cl[cl["Alto valor en riesgo"]][["ID Cliente", "Segmento", "Ciudad", "Compras 90d", "Gasto 90d COP",
"Dias Ultima Compra"]])

# No clasificables
print()
print("No clasificables (faltan datos esenciales):")
print(cl[cl["Categoria"] == "No clasificable"][["ID Cliente", "Segmento", "Compras 90d", "Gasto 90d COP",
"Dias Ultima Compra"]])

# Gasto por ciudad y categoría por segmento (contexto)
print()
print(cl.groupby("Ciudad")["Gasto_90d_COP"].agg(["count", "sum"]))
print()
print(pd.crosstab(cl["Segmento"], cl["Categoria"]))
```

Ejercicio 4 — Gráficos

```
# Gráfico 4: barras de clientes por categoría
orden = ["Alto valor", "Otros", "En seguimiento", "No clasificable"]
conteo = cl["Categoria"].value counts().reindex(orden)
colores = [AZUL, GRIS, NARANJA, "#c9c9c4"]
fig, ax = plt.subplots(figsize=(6.5, 4))
barras = ax.bar(conteo.index, conteo.values, color=colores, width=0.6)
for b, x in zip(barras, conteo.values):
    ax.text(b.get_x() + b.get_width() / 2, x + 0.3, str(int(x)), ha="center", fontsize=10)
ax.set title("Clientes por categoría")
ax.set ylabel("Cantidad de clientes"); ax.set ylim(0, conteo.max() * 1.2)
plt.tight_layout(); plt.savefig("graficos/g4_clientes_categoria.png"); plt.show()

# Gráfico 5: gasto total por categoría
gasto = cl.groupby("Categoria")["Gasto_90d_COP"].sum().reindex(orden)
fig, ax = plt.subplots(figsize=(6.5, 4))
barras = ax.bar(gasto.index, gasto.values / 1e6, color=colores, width=0.6)
for b, x in zip(barras, gasto.values / 1e6):
    ax.text(b.get x() + b.get width() / 2, x + 0.1, f"${x:.2f} M", ha="center", fontsize=10)
ax.set title("Gasto total (90 días) por categoría")
ax.set_ylabel("Millones de COP"); ax.set_ylim(0, (gasto.max() / 1e6) * 1.2)
plt.tight layout(); plt.savefig("graficos/g4 gasto categoria.png"); plt.show()

# Gráfico 6: dispersión compras vs gasto.
# Muchos clientes tienen exactamente los mismos valores, así que muevo un poquito los puntos (jitter)
# solo para que se vean todos. Los valores reales no cambian.
rng = np.random.default_rng(1)
p = cl.dropna(subset=["Compras 90d", "Gasto 90d COP"]).copy()   # C021 y C032 no se pueden graficar
fig, ax = plt.subplots(figsize=(7, 4.5))
estilos = {"Alto valor": AZUL, "Otros": GRIS, "En seguimiento": NARANJA}
for cat, col in estilos.items():
```

```
    s = p[p["Categoria"] == cat]
    ax.scatter(s["Compras 90d"] + rng.uniform(-0.12, 0.12, len(s)),
               s["Gasto_90d_COP"] / 1000 + rng.uniform(-6, 6, len(s)),
               s=45, color=col, edgecolor="white", linewidth=0.8, label=f"{cat} ({len(s)})")
ax.axhline(250, color="#555555", linestyle="--", linewidth=0.8)
ax.axvline(2.5, color="#555555", linestyle="--", linewidth=0.8)
ax.text(8.45, 262, "umbral gasto $250.000", ha="right", fontsize=8, color="#555555")
ax.text(2.55, 20, "umbral 3 compras", fontsize=8, color="#555555")
ax.set_title("Compras vs gasto en 90 días")
ax.set xlabel("Compras en 90 días"); ax.set ylabel("Gasto (miles de COP)")
ax.legend(frameon=False, fontsize=9, loc="upper left")
plt.tight_layout(); plt.savefig("graficos/g4_dispersion_compras_gasto.png"); plt.show()
print(f"En el gráfico: {len(p)} clientes (faltan C021 y C032 por datos inválidos)")
```

Anexo C. Bitácora de cambios de datos

Cada fila lista una anomalía, el tratamiento aplicado y el resultado. Ninguna fila fue eliminada y el CSV original no se modificó.

#	Archivo	ID	Campo	Valor original	Tratamiento	Resultado
1	Ventas	V009	Region	«␣norte␣»	Recortar + formato título	Norte
2	Ventas	V028	Canal	«web␣»	Recortar + formato título	Web
3	Ventas	V020	Ingreso_Registrado	$295.000 (esperado $270.000)	Se marca con Diferencia; el valor no se modifica	Diferencia = +$25.000
4	VidaPlus	A006	Sede	«centro␣»	Recortar + formato título	Centro
5	VidaPlus	A018	Espera_Min	(vacío)	Se deja vacío; Intervalo = «Sin clasificar»	null
6	VidaPlus	A026	Satisfaccion_1a5	6	Inválido (fuera de 1–5) en Satisfaccion_Valida	null
7	RutaExpress	E010	Zona	«␣urbana␣»	strip() + title()	Urbana
8	RutaExpress	E022	Costo_COP	−5.000	NaN; Costo_invalido = True	NaN
9	RutaExpress	E030	Dias_Reales	(vacío)	Retraso_Dias = NaN; Tiempo_invalido = True	NaN
10	Clientes	C011	Ciudad	«bogotá␣»	strip() + title()	Bogotá
11	Clientes	C021	Gasto_90d_COP	(vacío)	NaN; Gasto_faltante = True → No clasificable	NaN
12	Clientes	C032	Compras_90d	−1	NaN; Compras_invalida = True → No clasificable	NaN

Anexo D. Archivos depurados y entregados

Archivo	Contenido
RutaExpress_Limpio.csv	36 filas, 10 columnas. Zona estandarizada; Costo_COP sin negativos (E022 = vacío); columnas nuevas Costo_invalido, Retraso_Dias, Tiempo_invalido.
NovaMarket_Clientes_Limpio.csv	36 filas, 10 columnas. Ciudad estandarizada; columnas nuevas Compras_invalida, Gasto_faltante, Categoria y Alto_valor_en_riesgo.
Notebook_Ej3_Ej4.ipynb	Código de los ejercicios 3 y 4 con comentarios y salidas ejecutadas.
CSV originales (sin modificar)	01_NovaMarket_Ventas.csv, 02_VidaPlus_Atenciones.csv, 03_RutaExpress_Envios.csv, 04_NovaMarket_Clientes.csv.